Attention and transformers

IMDb

LECTURE 18 | GÉRON CH. 14–15

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Lecture 16 ended with three things recurrence cannot do, and Lecture 17 built a recurrent classifier that ran into all of them:

- The computation is **sequential** along the sequence
- Everything remembered must fit through **one fixed-width state**
- Position 400 is four hundred steps from position 2, however good the gates

> **ONE CHANGE REMOVES ALL THREE**
>
> Let every position compute a weighted sum over *every* position directly, with the weights learned from the content. No recurrence, no single state, and every pair of positions one step apart.

Today

1. **The mathematics** — scaled dot-product attention, and why the scores are divided by $\sqrt{d_k}$ (25 min)
2. Multi-head attention, and positional encoding (20 min)
3. **The method** — the encoder-decoder architecture and its three families (20 min)
4. Fine-tuning a pretrained transformer, against Lecture 17's model (20 min)

THE MATHEMATICS

Scaled dot-product attention

25 minutes · examinable

The question

Every position needs a summary of the whole sequence, weighted by *relevance* — and relevance has to be learned, not designed.

> **THE SHAPE OF THE ANSWER**
>
> Output at position i is $\sum_j w_{ij} \mathbf{v}_j$: a weighted sum of values, one per position. Everything below is about where w_{ij} comes from and why it takes the form it does.

Note what is already gone: there is no recurrence in that expression, so nothing forces the positions to be computed in order.

Queries, keys and values

1. [1] Project each position three ways, with learned matrices: $\mathbf{q}_i = \mathbf{W}_Q \mathbf{x}_i, \mathbf{k}_j = \mathbf{W}_K \mathbf{x}_j, \mathbf{v}_j = \mathbf{W}_V \mathbf{x}_j$.

   A query is what this position is looking for; a key is what that position offers; a value is what it contributes if chosen

2. [2] Score every pair by how well the query matches the key: $s_{ij} = \mathbf{q}_i^\mathsf{T} \mathbf{k}_j$.

   A dot product, because it is the cheapest similarity that a matrix multiply computes for all pairs at once

3. [3] Turn the scores into weights that sum to one: $w_{ij} = \mathrm{softmax}_j(s_{ij})$.

   Lecture 17's softmax, applied along the sequence rather than across classes

4. [4] And take the weighted sum: $\mathbf{z}_i = \sum_j w_{ij} \mathbf{v}_j$.

   In matrix form for all positions at once: $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\mathsf{T})\mathbf{V}$

Why divide by $\sqrt{d_k}$ — the variance

Take $\mathbf{q}$ and $\mathbf{k}$ to have independent components with mean 0 and variance 1. Then

1. [5] $s = \mathbf{q}^\top \mathbf{k} = \sum_{d=1}^{d_k} q_d k_d$ is a sum of d_k independent zero-mean terms.
   Each term has variance $\mathrm{Var}(q_d)\mathrm{Var}(k_d) = 1$

2. [6] So $\mathrm{Var}(s) = d_k$, and s has standard deviation $\sqrt{d_k}$.
   The scores grow with the dimension — for $d_k = 64$, typical scores are already of order 8

Why divide by $\sqrt{d_k}$ — the consequence

1. [7] Softmax of large scores saturates: one weight goes to 1, the rest to 0, and its gradient goes to zero with them.

   Exactly the vanishing gradient of Lecture 11, arriving through a different door

2. [8] Divide by $\sqrt{d_k}$ and the variance returns to 1, whatever the dimension:

   $$\text{Attention} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\mathsf{T}}{\sqrt{d_k}}\right)\mathbf{V}.$$

   The scale is not a hyperparameter — it is the number that makes the score distribution independent of d_k

Derive the variance of the score and state what the division fixes.

EXAMINABLE

What the derivation buys you

- Attention is **three projections, a scaled dot product, a softmax and a weighted sum** — no recurrence anywhere
- Every pair of positions interacts directly: the path length between any two is **1**, against $O(T)$ for a recurrent net
- All positions compute at once, so the sequence axis parallelises
- The cost is $O(T^2)$ in time and memory — quadratic in sequence length, which is the price paid for the constant path length

> **THE TRADE, STATED PLAINLY**
>
> Recurrence is linear in T and sequential, with a long gradient path. Attention is quadratic in T and parallel, with a path of length one. On the sequence lengths people actually use, the second wins — and when it does not, the T^2 term is what has to be approximated.

Multi-head attention

One attention layer computes one notion of relevance. Run h of them in parallel on *different learned projections* of the same input, and concatenate.

- Each head has its own $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, of width d_{model}/h — so h heads cost the same as one wide one
- Different heads learn different relations: adjacency, agreement, coreference. Nothing tells them to; the division of labour is learned
- Concatenate and project back to d_{model}

The argument is the ensemble argument of Lecture 7 in a new place: several estimators of the same thing beat one, provided their errors are not identical — and different projections are what stops them being identical.

Positional encoding

THE PROBLEM ATTENTION CREATES

$\mathrm{softmax}(\mathbf{QK}^\top)\mathbf{V}$ is **permutation equivariant**: shuffle the input positions and the outputs shuffle with them, unchanged. So the model literally cannot tell *"the film was not good"* from *"good, the film was not"*.

Recurrence got order for free, because it processed the sequence in order. Attention has to be *told*: add a position-dependent vector to each embedding before the first layer.

- **Sinusoidal** — fixed, and extends to lengths never seen in training
- **Learned** — an embedding table indexed by position; simpler, and bounded by the longest training sequence
- **Relative** — encode the *distance* between two positions in the score, which is what most current models do

THE METHOD

Encoder, decoder, and the three families

20 minutes · examinable

One block

A transformer block is attention plus a small feed-forward network, each wrapped the same way:

1. Multi-head self-attention over the whole sequence
2. Add the input back (a residual connection) and normalise
3. A position-wise feed-forward network — the same two-layer MLP applied at every position independently
4. Add and normalise again

> **WHY THE RESIDUAL, AND WHY THE NORM**
>
> Both are Lecture 11, unchanged. The residual gives the gradient an additive path through depth — the same argument as an LSTM gate, one level up. The layer normalisation keeps the variance from drifting as blocks stack. A transformer is deep, and it is deep only because of these two.

Masking, and why the decoder needs it

A decoder generating text must not look at what it has not written yet. So set $s_{ij} = -\infty$ for $j > i$ before the softmax.

$$w_{ij} = \mathrm{softmax}_j\left(\frac{s_{ij}}{\sqrt{d_k}} + m_{ij}\right), \quad m_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

$e^{-\infty} = 0$, so those positions get exactly zero weight and contribute nothing — and, crucially, no gradient either.

✗ This is Lecture 15's causality condition again, enforced in the architecture rather than in the data split. An unmasked decoder is a model that has read the answer.

Three families

Family	Attention	Good at
Encoder-only BERT-like	bidirectional	classification, tagging, retrieval — anything where the whole input is available
Decoder-only GPT-like	masked, causal	generation, and everything phrased as generation
Encoder–decoder T5-like	both, plus cross-attention	translation, summarisation — a sequence in, a different sequence out

The choice is the same question as bidirectional layers in Lecture 16: *at prediction time, do I have the whole input?* Today we classify a review we can see in full, so encoder-only is right.

What a pretrained transformer already knows

It was trained on a large corpus to fill in masked tokens or predict the next one. Nothing about sentiment, and yet:

- Syntax, agreement, and long-range dependence — because predicting a missing word requires them
- A great deal of lexical semantics, since words that fill the same slots get similar representations
- Enough about discourse that a classification head on top needs very few labels

Which is Lecture 13's argument, word for word, on a different modality: **do not learn what somebody has already learned for you.** The backbone is the transformer; the head is small and random; the labels are scarce.

Attention in ten lines

```
def attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)   # the derivation
    if mask is not None:
        scores = scores.masked_fill(mask, float("-inf"))
    w = scores.softmax(dim=-1)
    return w @ V, w
```

Six lines, and every one of them is a step of the derivation. The rest of a transformer — the heads, the residuals, the feed-forward, the normalisation — is arrangement around this function.

Which is worth saying plainly: the architecture that changed the field is, at its centre, a similarity, a softmax and a weighted average.

Checking it against the library

- Same inputs, same seed, compare to `F.scaled_dot_product_attention` — the two should agree to floating-point tolerance
- If they do not, the usual causes are a transposed key matrix, a mask applied after the softmax instead of before, or a missing scale
- Each of those **runs**. Only the comparison finds them

> **WHY IMPLEMENT IT AT ALL, IF A LIBRARY HAS IT**
>
> The same reason Lecture 2 verified the normal equation numerically and Lecture 10 checked autograd against a hand derivative: a formula you have implemented and tested is one you can reason about when the model misbehaves. A formula you have only read is not.

Where the parameters are

Component	Parameters
Attention, per layer	$4\,d_{\text{model}}^2$ — the four projections $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$ and the output
Feed-forward, per layer	$8\,d_{\text{model}}^2$ — usually a 4× expansion and back
Embeddings	$\lvert V \rvert \times d_{\text{model}}$ — often a large fraction of a small model

✗ Two thirds of the parameters in a transformer block are in the *feed-forward*, not in the attention. The attention is what makes it work and it is the minority of the weights — a useful corrective to the name.

Same habit as Lecture 9: count the parameters before you fit anything, and know where they are.

The notebook for this lecture

Open Lecture 18 in Colab

- **Runs on free CPU.** A small pretrained transformer, a subsampled corpus, and a short fine-tune
- Attention implemented from the derivation in about ten lines, and checked against the library's
- The $\sqrt{d_k}$ scaling removed, and the softmax saturation that follows, measured

> **WHAT TO CHANGE**
>
> Shuffle the token positions of an input and run it through attention with and without positional encoding. Without, the output is the same multiset; with, it is not.

The model we borrow

Property	Value
Parameters	66,955,010
Vocabulary — the same one we used last lecture	30,522
Embedding width	768
Pretraining task	predict masked words

Last lecture we took 30,522 × 768 numbers out of this model and threw the rest away. The rest is where the language is.

Fine-tuning, precisely

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2)
# every weight is pretrained EXCEPT the classification head, which is random

opt   = torch.optim.AdamW(model.parameters(), lr=2e-5)
lossf = nn.CrossEntropyLoss()          # logits in, as always
```

- A new linear head, randomly initialised, on top of a pretrained body
- A learning rate around 100 times smaller than we used from scratch — large steps destroy what was pretrained
- One pass over the data, not four

How much is the head, and how much is the body?

Run the same backbone with its untrained head and no fine-tuning at all:

Model	Test accuracy
✗ Pretrained body, random head, no training	**✗ 40.1%**
Always one class	50.0%

Measured on 5,000 test reviews sampled at random, 49.8% of them positive. Note it is *below* the majority class: an untrained head is an arbitrary hyperplane, and which side it puts each class on is a coin toss. The pretraining on its own predicts nothing about sentiment, because nothing has told it what the two output units mean.

Everything that follows is what one pass of fine-tuning does with a representation it did not have to build.

What it costs

Setting	Value
Reviews used to fine-tune	20,000
Epochs run, and the one validation chose	2, 1
Batch size, learning rate	16, 0.00002
Training wall clock	346 s
Scoring 25,000 test reviews	133 s

Measured on an Apple MPS backend, in `float32`. The number of epochs is a hyperparameter, so it is chosen on the 5,000-review validation split, exactly as in Lecture 2 — never on the test set.

The scoring time is the median of 5 passes, the same protocol as Lecture 21's cost table. It is also the least reproducible number in this deck: the five passes ran from 103 s to 160 s. Quote it to two figures or not at all.

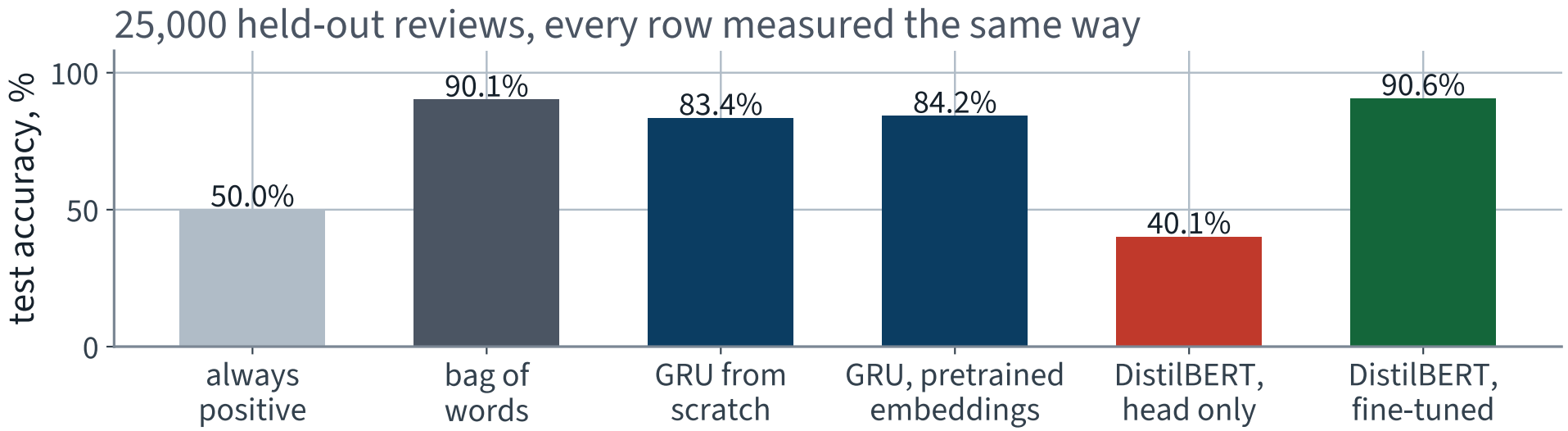
The result

Model	Test accuracy
Always one class	50.0%
tf-idf + logistic regression	90.1%
GRU from scratch	83.4%
GRU, pretrained embeddings	84.2%
✓ DistilBERT, fine-tuned	**90.6% ✓**

✓ 7.18 points over the model you built.

And only 0.45 points over the bag of words — which is the second result of the afternoon, and the less comfortable one.

Everything, measured the same way



Five of the six bars are on the same 25,000 held-out reviews, with the same metric, computed by the same function.

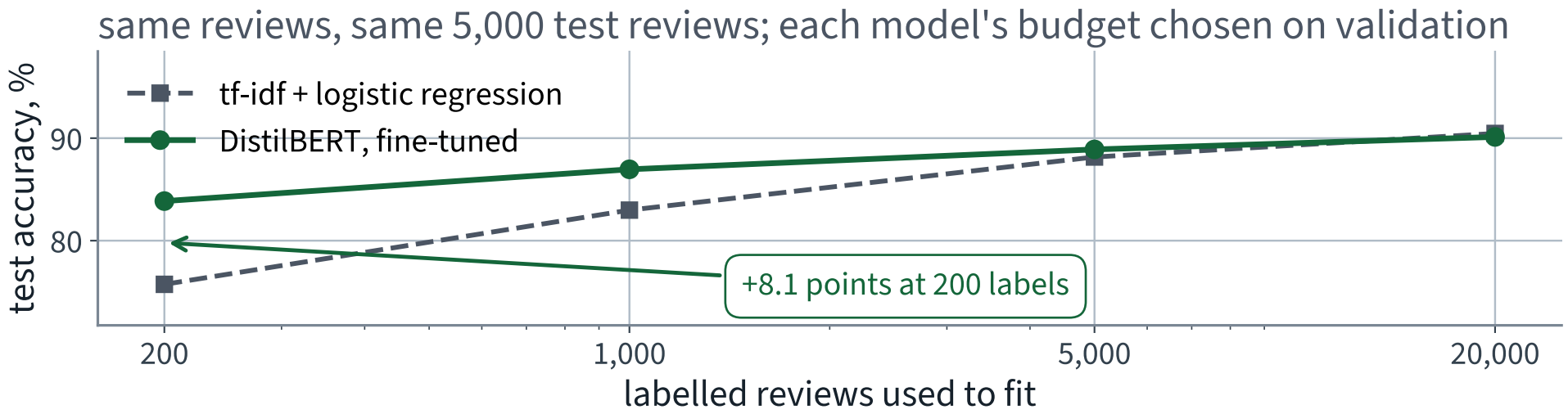
Say it in errors, not in points

Model	Reviews misclassified, out of 25,000
GRU from scratch	✗ 4,146
✓ DistilBERT, fine-tuned	2,352 ✓

43%

of the mistakes removed. Percentage points flatter a model near the top of the range; the error count is the number the desk actually cares about, because each one is a review someone has to fix by hand.

Where the pretrained model earns its place



✓ The gap is largest where labels are scarcest — which is the situation every real feedback desk starts in.

Label efficiency, measured

Labelled reviews	Bag of words	DistilBERT	Gap
200	75.7%	**83.9% ✓**	**✓ 8.12**
1,000	83.0%	**87.0% ✓**	**✓ 3.98**
5,000	88.2%	**88.9% ✓**	**✓ 0.74**
20,000	90.5%	90.1%	-0.34

Each model's budget is **chosen on the validation split at every size** — the number of optimisation steps for the transformer, the regularisation strength for the linear model. Fixing the epochs instead would give the 200-review model 13 steps, and fixing the steps would undertrain the 20,000-review one; measured, those two protocols disagree about the *sign* at 20,000 labels.

Scored on 5,000 test reviews rather than all 25,000, to keep the sweep inside a lecture — sampled at random, because the corpus ships all the positives first.

The pretraining is worth thousands of labels. That is the number to quote to whoever is paying for the annotation.

And the axis the accuracy plot does not have

Lecture 21 timed the anchors because the brief said “within the hour”. Put the two together and the trade is visible:

Model	Accuracy	Scoring 25,000	Per review
tf-idf + logistic regression	90.1%	**3.6 s ✓**	0.14 ms
GRU from scratch	83.4%	6.3 s	0.25 ms
✓ DistilBERT, fine-tuned	**90.6% ✓**	**✗ 133 s**	**✗ 5.30 ms**

0.45 points, for 37 times the cost per review.

Which is a decision, not a result. At 20,000 labelled reviews the desk would probably make it against us; at 200 it would not, and the previous slide says by how much.

What did the work

Changed	Unchanged
the whole model, not just the table	the corpus, the split, the metric
attention instead of recurrence	the loss — still logits in
a much smaller learning rate	the training loop, line for line
one epoch instead of four	the evaluation function

Every row on the right is a decision you do not have to re-make when the model changes. That is what the discipline of Part I bought.

The desk wanted two more things

“Find me the ones like this one.”

“And tell me what people keep complaining about.”

Neither is classification. Both fall out of the same pretrained models, because what those models produce is a **representation**, and a label was only ever one thing to do with it.

Search, then grouping. Fifteen minutes, and no training at all.

One vector for a whole review

```
1  m   = AutoModel.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")
2  tk  = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")
3
4  enc  = tk(texts, truncation=True, max_length=256, padding=True,
5            return_tensors="pt")
6  out  = m(**enc).last_hidden_state                    # (B, T, 384)
7  mask = enc["attention_mask"].unsqueeze(-1).float()
8  vec  = (out * mask).sum(1) / mask.sum(1)             # mean over REAL tokens
9  vec  = torch.nn.functional.normalize(vec, dim=1)     # onto the unit sphere
```

✗ Line 8 is last lecture's padding bug in its other costume. Divide by `out.shape[1]` instead of by `mask.sum(1)` and every short review is scaled toward zero.

Why normalise

On the unit sphere the dot product *is* the cosine:

$$\langle \hat{\mathbf{u}}, \hat{\mathbf{v}} \rangle = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = \cos\theta \in [-1, 1]$$

- Length carries no meaning here, and a long review would otherwise dominate every ranking
- Searching 5,000 reviews becomes one matrix product

Normalisation onto the sphere returns as the first line of thread 12, where it is doing rather more work.

Semantic search, in two lines

```
sims = query_vec @ corpus_vecs.T          # (1, N) cosine similarities
top  = np.argsort(-sims[0])[:3]
```

5,000 negative reviews, embedded once in 11 seconds, 384 dimensions each. No index, no training, no labels.

> **COMPARE WITH WHAT IT REPLACES**
>
> A keyword search over the same corpus, using the tf-idf vectoriser from the previous lecture. Same corpus, same queries, different notion of “similar”.

One query, two searches

Query: *“the acting was wooden and unconvincing”*

> **NEAREST BY EMBEDDING — COSINE 0.638**
>
> I caught this film late at night on HBO. Talk about wooden acting, unbelievable plot, et al. Very little going in its favor. Skip it.

Nearest by keyword overlap — tf-idf cosine 0.229:

“This is by far the worst non-English horror movie I've ever seen. The acting is wooden, the dialogues are simply stupid and the story is totally...”

Both found it. The scores are not comparable across the two methods — different spaces — so the interesting question is the next slide's.

Where they disagree, over three queries

Query	Shared in the top 3
“the acting was wooden and unconvincing”	0 of 3
“the sound mix made the dialogue impossible to follow”	0 of 3
“far too long, it should have ended an hour earlier”	0 of 3

Not one document in common, on any of the three. Keyword search can only retrieve documents that reuse the query's words, and a complaint rarely uses the desk's vocabulary; the embedding was trained so that sentences with the same meaning land near each other, whatever words they use.

✗ That is a difference, not a victory — and nothing here measures which list is *better*. Judged by eye, on one of these three the keyword search returns the more relevant review. Cosine similarity also has no notion of negation, so *the sound was perfect* and *the sound was not perfect* sit close together. If you deploy this, you owe yourself labelled queries and a retrieval metric — which is Lecture 23.

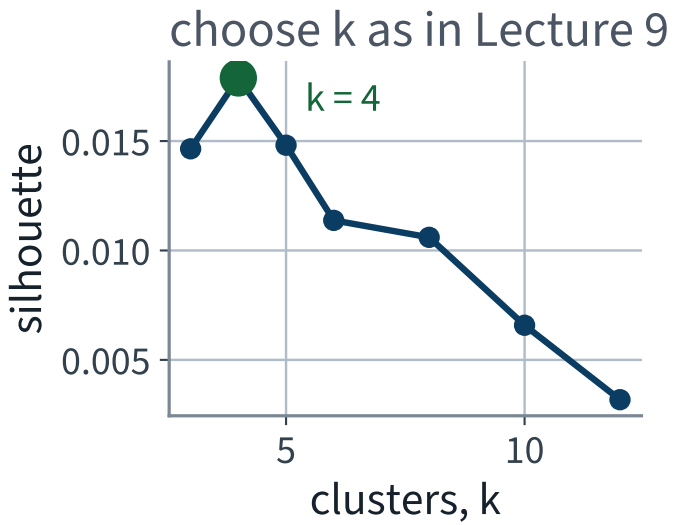
“What do people keep complaining about?”

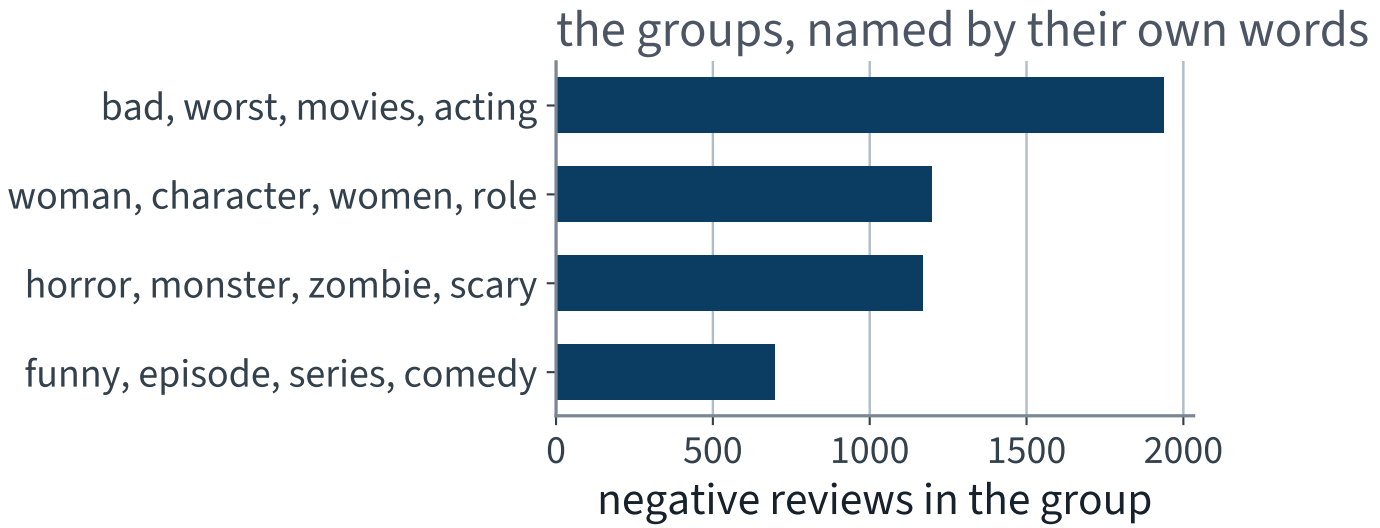
Cluster the 5,000 negative reviews in the same embedding space. This is Lecture 9, unchanged — k-means, and the choice of k by silhouette rather than by eye.

```
for k in (3, 4, 5, 6, 8, 10, 12):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(V)
    print(k, silhouette_score(V, km.labels_, sample_size=3000,
                              random_state=42))
```

The vectors are on the unit sphere, so Euclidean k-means and cosine k-means order the same points the same way.

The groups, and how many





The silhouette is low in absolute terms — complaint text does not fall into well-separated balls — and the groups are still legible.

The three largest groups

Reviews	Terms most characteristic of the group
1,936	bad, worst, movies, acting
1,198	woman, character, women, role
1,170	horror, monster, zombie, scary

The terms are not the cluster centres. They are the words whose average tf-idf weight is highest *inside* the group relative to outside it — a group is named by what makes it different, not by what is common everywhere.

4 groups in total, chosen by a silhouette of 0.0179.

What the desk actually gets

They asked for	They get
a complaint flag	90.6% accurate, 133 s for 25,000 reviews
“find me ones like this”	cosine search over 5,000 vectors
“what recurs”	4 groups, each named by its own words

Three deliverables, one pretrained representation, and one afternoon. None of the three required a labelled example of the thing being asked for except the first.

One more prompt, and this is the one to remember

> **THE PROMPT**
>
> *“Build a scikit-learn pipeline that vectorises the reviews with tf-idf and classifies them with logistic regression, and report the test accuracy.”*

Under-specified in the same place as Lecture 1's, and this time the object being fitted is not a scaler. Read the code and name what is fitted, and on what.

The code it returns

```
vec = TfidfVectorizer(min_df=1, ngram_range=(1, 2))
X   = vec.fit_transform(all_texts)            # every document
y   = all_labels

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

clf = LogisticRegression(max_iter=2000, C=4.0).fit(X_tr, y_tr)
print(f"accuracy {(clf.predict(X_te) == y_te).mean():.3f}")
```

It runs, it splits, it never fits the classifier on test rows, and it reports a plausible number.

Reviewer question 2: what was fitted, and on what?

`TfidfVectorizer.fit` learns two things from the documents it is given, and both come from the test rows here:

- The **vocabulary** — which terms get a column at all. Terms that occur only in the test documents get their own feature
- The **inverse document frequencies** — the weight on every column, computed over the whole corpus

> **WHY THIS IS WORSE THAN LECTURE 1'S SCALER**
>
> A scaler learns two numbers per column from an *invertible affine* map. A vectoriser learns **which columns exist**, and that is not a transformation of the features — it is a choice of feature space, made with the answers in the room.

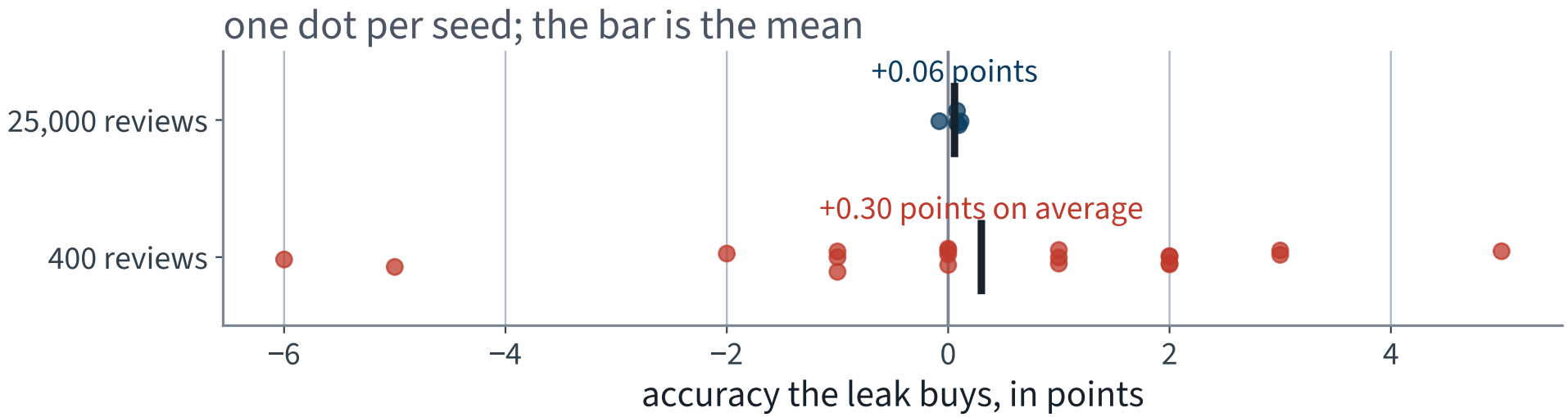
So measure it — twice

The same experiment at two corpus sizes, because the answer depends on the size and that is the lesson:

- **25,000 documents** — the whole of IMDb, 5 seeds
- **400 documents** — the size of a real feedback corpus in its first month, 20 seeds

Leaky and honest are run on identical splits with identical seeds, so the difference is the leak and nothing else.

What the leak buys, per seed



Two corpora, the same three lines of code, and two different verdicts.

The leak, measured

Corpus	Leaky	Honest	Difference	sd over seeds
25,000 reviews	90.28%	90.22%	0.06 pts	0.07%
400 reviews	75.90%	75.60%	0.30 pts	✗ 2.53%

Read the last column before the one beside it.

On the small corpus the mean gap is 0.30 points and the spread across seeds is 2.53% — eight times larger. The leak wins on 10 of 20 seeds: exactly half. **This is a null result.**

Why it is so cheap — and this is the useful part

The leaky vocabulary really is bigger: 60,000 columns against 54,074. So why does it buy nothing?

- A term that occurs only in test documents has an **all-zero column** in the training rows. Logistic regression gives it coefficient exactly zero — there is no gradient to move it
- What actually leaks is the inverse document frequencies, and an idf is an average over the whole corpus. Removing a quarter of the documents barely moves it
- So the leak changes the *scale* of features the model already had, and a regularised linear model is nearly indifferent to that

> **THE DECISION RULE, AND IT IS NOT ABOUT THE SIZE**
>
> Fit the vectoriser inside the pipeline anyway. Not because this leak is expensive — measured, it is not — but because **the next fitted object in the pipeline may not have that property**, and you will not re-derive the argument each time.

Three leaks, three small numbers

Lecture 1's scale-before-split leak was worth about a dollar. Lecture 12's per-batch mean was worth 0.382 accuracy points. Today's vectoriser leak is worth nothing you can measure.

> **AND THAT IS NOT A REASON TO RELAX**
>
> A leaked score and an honest score can be identical, so you cannot tell from the number which one you are holding. The procedure is what separates them — which is why the procedure is what gets checked in review, and why the next slide matters more than this one.

The text leak that really is expensive

The same document on both sides of the split.

No pipeline protects you from this: every object is fitted correctly, on training rows only, and the test row is still one the model has memorised. On scraped or user-submitted text it is the normal case, not the pathological one — a resubmit button is enough.

```
norm = lambda s: " ".join(s.lower().split())
train_norm = set(norm(s) for s in train_x)
dupes = sum(1 for s in test_x if norm(s) in train_norm)
```

First, check the corpus you have

On the official IMDb split	Count
✗ Test reviews also present in training	✗ 123
✗ Duplicate reviews within training	✗ 96

Not zero. The corpus authors separated the *films* between train and test, and said so; they did not deduplicate the *reviews*. Three lines to run, and you now know rather than assume.

123 out of 25,000 is too few to move our numbers. It is also exactly the check nobody ran, on the corpus everybody uses.

A feedback corpus that was never deduplicated

123 duplicates in 25,000 is not enough to measure a cost with, so build a corpus where it is: 1,500 reviews, a third of them submitted twice, 1,950 rows in all. Then split two ways and fit the same honest pipeline inside each.

```
groups = np.concatenate([np.arange(n_unique), duplicated_ids])
tr, te = train_test_split(np.arange(len(X)), test_size=0.25,
                          random_state=seed, stratify=y)

gss = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=seed)
tr2, te2 = next(gss.split(np.arange(len(X)), y, groups=groups))
```

✗ Under the random split, 35% of the test rows have a copy of themselves in the training set.

The same pipeline, two splits

✗ Nothing was fitted on the test set in either bar. The difference is entirely which rows the split happened to separate.

The duplicate leak, measured

Split	Reported accuracy	Spread over 10 seeds
✗ random over rows	✗ 90.00%	1.87%
✓ grouped by entry	82.17% ✓	2.88%

7.83 points

The leak wins on 10 of 10 seeds. Compare with the vectoriser leak two slides ago: **the object that was fitted wrongly cost far less than the rows that were split wrongly.**

The silent-failure catalogue so far

Failure	Diagnosed in	Measured cost
Scaler fitted before the split	Lecture 2	about a dollar
Accuracy under class imbalance	Lecture 4	a useless model above 90%
Missing `zero_grad()`	Lecture 12	76.7 points
Random split on a time series	Lecture 20	an unreproducible score
✗ Summarising at the padded last position	**✗ Lecture 21**	**✗ 18.24 points**
✗ Softmax before the loss	**✗ today**	**✗ 1.67 points**
Vectoriser fitted before the split	today	0.30 points — not distinguishable from zero
✗ Duplicate documents across the split	**✗ today**	**✗ 7.83 points**

Is 90.6% good?

Against the desk's own criterion: 2,352 reviews out of 25,000 are routed wrongly, and nobody sees them again. Whether that is acceptable depends on what happens to a missed complaint, and we have not asked.

> **WHAT A DEFENSIBLE REPORT SAYS**
>
> Quote the error count, not the percentage. Say which reviews are misread — short ones, sarcastic ones, ones whose verdict is in the last clause. Offer an abstain band around a probability of one half, and let the desk choose how wide it is.

The model is not the deliverable. The routing decision is.

What carried over unchanged

From	Still true
Lecture 1	compute the trivial baseline, and a serious one, before committing
Lecture 2	split first; fitted objects see training rows only
Lecture 9	choose k by silhouette, not by eye
Lecture 10	PCA when a representation is wider than the slot it must fit
Lecture 12	own the loop; a metric is counted over the set
Lecture 16	a pretrained representation beats a bigger architecture

Where this went past the book

> **NOT EXAMINABLE**
>
> The `transformers` library, the model hub, and the specific checkpoints used today. Examinable: subword tokenisation, embeddings, attention and the transformer block, fine-tuning a pretrained model, sentence embeddings and their use for retrieval and clustering — all of Chapters 14 and 15.

Also outside the book, and worth one sentence: the models used today are small by current standards, and everything in this lecture applies unchanged to models a thousand times larger, with a different bill.

In the book

Topic	Chapter
Cross-entropy and softmax output layers	3, 9
Sentiment analysis, subword tokenisation	14
Reusing pretrained embeddings and models	14
Attention and the transformer architecture	15
Fine-tuning a pretrained language model	15
k-means and silhouette scores	8

Self-attention against cross-attention

	Queries from	Keys and values from	Used in
Self-attention	the sequence	the same sequence	every block of every family
Cross-attention	the decoder	the encoder's output	encoder–decoder models only

The arithmetic is identical — the same scaled dot product, the same softmax. Only where $\mathbf{Q}$ and $\mathbf{K}$ come from changes, which is why one implementation serves both.

In a translation model, cross-attention is the place the output sentence looks at the input one, and its weights are often close to a word alignment without anyone having asked for that.

The quadratic cost, and what is done about it

$\mathbf{QK}^\mathsf{T}$ is $T \times T$. At $T = 512$ that is 262,144 scores per head per layer; at $T = 8{,}192$ it is 67 million.

- **Sparse patterns** — attend to a local window plus a few global positions, making the cost linear
- **Low-rank or kernel approximations** — approximate the softmax so the product never has to be formed
- **Better implementations** — the exact same computation, tiled so it never writes the $T \times T$ matrix to memory. This is what most systems actually use

✗ Note which of those changes the mathematics and which does not. The third is an engineering result, and it moved the practical limit further than either of the first two.

Reading an attention map, carefully

Attention weights are easy to plot and easy to over-read.

> **WHAT THEY ARE NOT**
>
> They are **not an explanation**. A weight says this value contributed to that output, not that the model's decision depended on it: the residual stream carries information around the attention, several heads disagree, and different weight patterns can give identical outputs.

Useful for forming a hypothesis; not evidence on its own. The test is the same as everywhere else in this course — remove the thing and measure whether the number moves.

Tokenisation, revisited

A transformer inherits Lecture 17's tokenizer, and now the choice bites harder:

- Sequence length is measured in *tokens*, and the cost is quadratic in it, so a wasteful tokenizer is expensive twice over
- The pretrained model's tokenizer is **part of the weights**. Use a different one and every embedding is indexed wrongly — the model runs, and produces nonsense
- Rare words split into several tokens, so text in a domain the tokenizer never saw is silently longer and more expensive

✗ Loading a checkpoint with the wrong tokenizer raises nothing. It is this lecture's version of Lecture 13's normalisation constants: preprocessing that ships *with* the weights.

The budget

What	Cost
Pretraining a transformer from scratch	**✗ out of reach here, and out of reach for most**
Fine-tuning all parameters	feasible on one accelerator for a small model
Fine-tuning the head only	**minutes on a CPU ✓**
Prompting a large model, no training	**no training at all, and no control ✓**

The middle two are Lecture 13's probe-then-tune ladder, unchanged. The bottom row is new, and it is a different kind of thing: you are not fitting anything, so there is nothing to evaluate on a validation split — which makes the evaluation question harder, not easier.

What we did not do

- **Train a transformer from scratch.** The compute is not the point; the architecture is
- **Instruction tuning and preference optimisation** — how a pretrained model becomes an assistant
- **Efficient attention in detail**, beyond naming the three approaches
- **Scaling laws** — how loss falls with parameters, data and compute together

Named so you recognise them, not examinable. What *is* examinable is the derivation and the three families.

BEYOND THE SYLLABUS, FOR CONTEXT

Vocabulary

logit	an unnormalised score, before softmax — Lecture 17
head	one attention computation, on its own projections
d_k	the width of the query and key vectors, and what the scale corrects for
causal mask	the $-\infty$ that stops a position seeing later ones
residual stream	the additive path each block writes into
context length	the longest sequence a model can attend over
cross-attention	queries from one sequence, keys and values from another

Scope

- the attention derivation including the $\sqrt{d_k}$ argument; multi-head attention and why it helps; why positional information must be added; the three families and how to choose; the causal mask — EXAMINABLE
- library APIs, checkpoint names, and the details of efficient attention — NOT EXAMINABLE — ENGINEERING
- pretraining, instruction tuning, scaling laws — BEYOND THE SYLLABUS

What we did

1. Derived attention as three projections, a scaled dot product, a softmax and a weighted sum — with no recurrence in it
2. Derived the $\sqrt{d_k}$: the score variance grows with the dimension, a saturated softmax has no gradient, and the division is exactly the correction
3. Saw multi-head attention as Lecture 7's ensemble argument on different learned projections
4. Saw that attention is permutation equivariant, so position has to be supplied — and three ways to supply it
5. Read the block: attention, residual, normalise, feed-forward, residual, normalise; and the mask that makes a decoder causal
6. Fine-tuned a pretrained transformer and compared it with Lecture 17's recurrent model on the same split

One line to keep

> **THE LINE**
>
> **Attention replaces a path of length T with a path of length one, and pays T^2 for it.**

Every other property follows. Parallel along the sequence, because nothing waits for the previous step. No fixed-width bottleneck, because there is no single state. Order must be supplied, because a set has none. And a quadratic bill, which is what the last decade of engineering has been spent reducing.

Before the next lecture

- Run the notebook. Remove the $\sqrt{d_k}$ and watch the attention weights collapse onto one position
- Part V begins next: **information retrieval**, then **recommender systems**. Four lectures outside the textbook, taught from the lecture notes, and fully examinable
- They are where the embeddings of the last two lectures earn their living: a query and a document in the same space is the whole of dense retrieval, and it is Lecture 20

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 18

five, in the style of the written paper

Exercises — Lecture 18

1 Scaled dot-product attention divides by $\sqrt{d_k}$. State what the variance of the unscaled dot product is, and what the softmax does without the division. [5]

2 A student adds a softmax to a model whose loss is `CrossEntropyLoss`. Give the floor the loss cannot go below on two classes, and the arithmetic behind it. [5]

3 A pretrained body is fine-tuned at 10^{-3}, the rate that worked for the from-scratch model. Predict what happens, and how quickly. [4]

Solutions on Lecture 19's deck.

Exercises — Lecture 18 (continued)

4 Two corpora, of 400 and 25,000 documents, each have a vectoriser fitted before the split. State which suffers more, and why. [4]

5 A corpus contains duplicate documents across the train/test split. State why no pipeline fixes it, and name the splitter that does. [4]

Solutions on Lecture 19’s deck.

THE FIVE SET AT THE END OF LECTURE 17

Solutions Lecture 17

not part of this lecture

Solutions — Lecture 17

1. Show that softmax is invariant to adding a constant to every logit, and state the numerical reason the…

✓ e^c cancels between numerator and denominator. Subtracting the max keeps every exponential below 1, and float32 overflows at about 88.7.

- The function ignores the shift; the arithmetic does not
- Moving to float64 only relocates the threshold to about 709

2. Derive $\partial L / \partial \mathbf{z} = \mathbf{p} - \mathbf{y}$ for cross-entropy on a one-hot target, and…

✓ $L = -z_c + \log \sum_j e^{z_j}$, whose derivative is $\mathbf{p} - \mathbf{y}$; each row sums to zero.

- The exponential of the true class cancels, so no chain rule through the softmax is needed
- A zero row sum is the derivative form of the shift invariance

Solutions — Lecture 17 (2)

3. A loss function is given probabilities rather than logits. State the two failure modes, and which one ships.

✓ Overflow to inf or nan, and a finite but wrong value. The quiet one ships.

- Taking the log of an underflowed probability is the loud failure
- A row whose true class has small probability loses precision silently

4. A model summarises a padded batch at out[:, -1, :]. State what that position holds for most reviews, and the…

✓ Padding. Encode the same review with two different amounts of padding and require the logits to agree.

- That position is real content only for the longest reviews
- An output-shape test cannot see this; an invariance test can

Solutions — Lecture 17 (3)

5. A word-level vocabulary has a 40% out-of-vocabulary rate over distinct test words. Explain why a larger…

✓ A word vocabulary is closed and language is not.

- Two words in five are seen once, so more capacity buys mostly those
- Subword tokenisation changes the closure property rather than the size

LECTURE 19 · LECTURE NOTES

Information retrieval: the lexical foundation

A query, a corpus, a ranking — and why it is not classification
The inverted index, term weighting, and BM25 derived from what each of its parameters is for
Evaluating a ranking: precision@k, MRR, and NDCG

Part V begins — beyond the textbook, and examinable